

Flutter - Quiz-App

In diesem Kapitel bauen wir gemeinsam eine einfache **Quiz-App** in Flutter. Die App besteht aus einzelnen Fragen zu Flutter und einer Zusammenfassung am Schluss, welche Fragen korrekt beantwortet worden sind. Danach kann man das Quiz erneut starten.

Die App wird aus mehreren Seiten bestehen und wir sehen uns an, wie man den Zustand der App verwalten kann, um zwischen den Seiten zu wechseln.

Neue Flutter-App erstellen

Erstelle ein neues Flutter-Projekt mit dem Namen `quiz_app`:

```
flutter create quiz_app
```

Öffne das Projekt in Visual Studio Code:

```
cd quiz_app  
code .
```

Falls du folgende Fehlermeldung bekommst ...

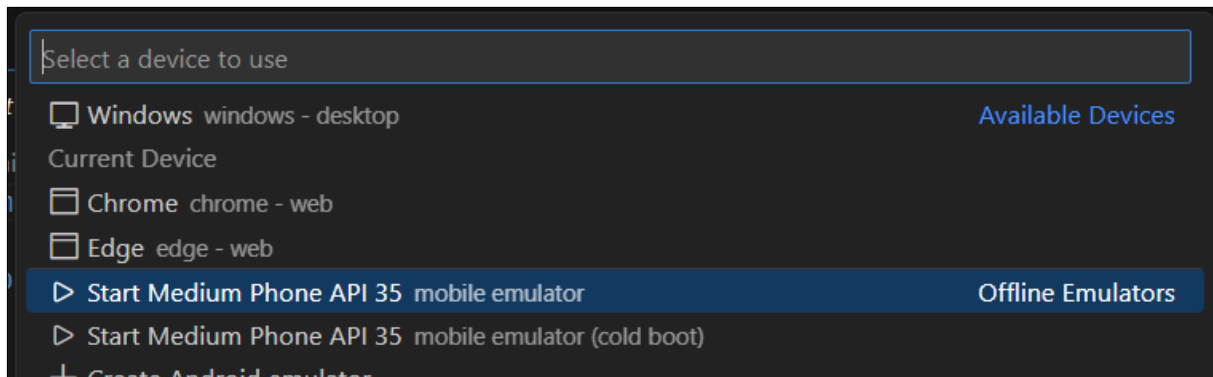
```
The configured version of Java detected may conflict with the Gradle  
→ version in your new Flutter app.
```

...

...dann öffne die Datei `android/gradle/wrapper/gradle-wrapper.properties` und ändere die Zeile `distributionUrl` auf:

```
distributionUrl=https\://services.gradle.org/distributions/gradle-8.3-all.zip
```

Öffne nun die Datei `lib/main.dart`, drücke [F1] und gib "Flutter: Select Device" ein. Wähle den passenden Emulator aus ...



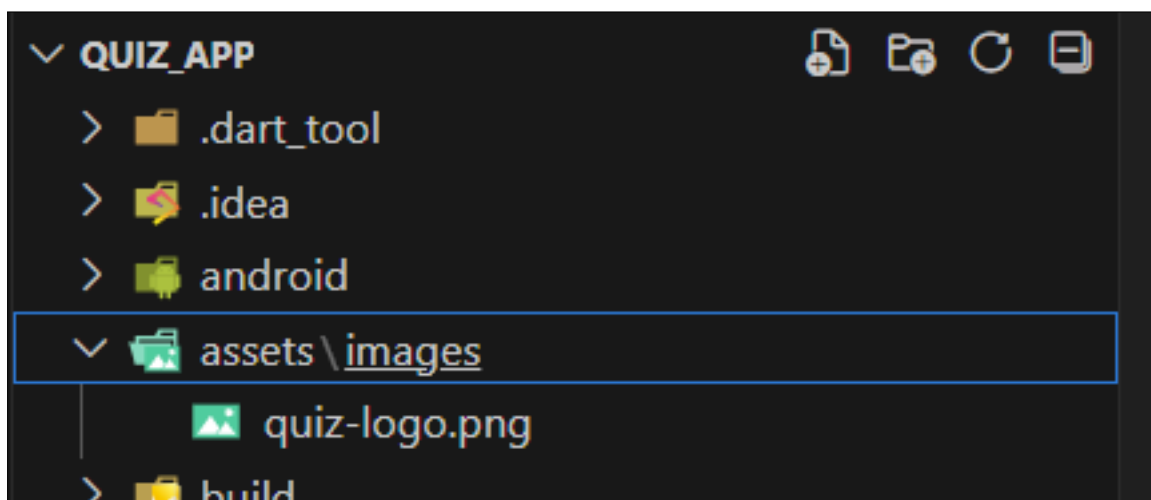
... und drücke [F5] um die App zu starten.

HINWEIS: Die Fehlermeldung *"Warning: SDK processing. This version only understands SDK XML versions up to 3 but an SDK XML file of version 4 was encountered. This can happen if you use versions of Android Studio and the command-line tools that were released at different times."* kannst du ignorieren.

Wenn alles klappt, stoppe die App wieder und entferne alles aus der `main.dart`-Datei. Wir starten wieder "from scratch".

Startseite erstellen

Lade das ZIP-File `assets.zip` aus dem Edu-Kurs herunter und entpacke es in deinem Projektverzeichnis. Du solltest danach ein Verzeichnis `assets` mit folgendem Inhalt haben:



Nun implementieren wir die Startseite unserer App. Füge dazu folgenden Code in die `main.dart`-Datei ein:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MaterialApp(home: Scaffold(body: )));
}
```

Für den body erstellen wir wieder ein eigenes "custom widget". Wir könnten das Scaffold-Widget auch in unser neues Widget einbauen, doch in diesem Beispiel verwenden wir die gleiche Hintergrundfarbe für alle Seiten.

Erstelle also eine neue Datei `lib/start_screen.dart` und füge folgenden Code ein:

```
import 'package:flutter/material.dart';

class StartScreen extends StatelessWidget {
  const StartScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return const Text('Start Screen');
  }
}
```

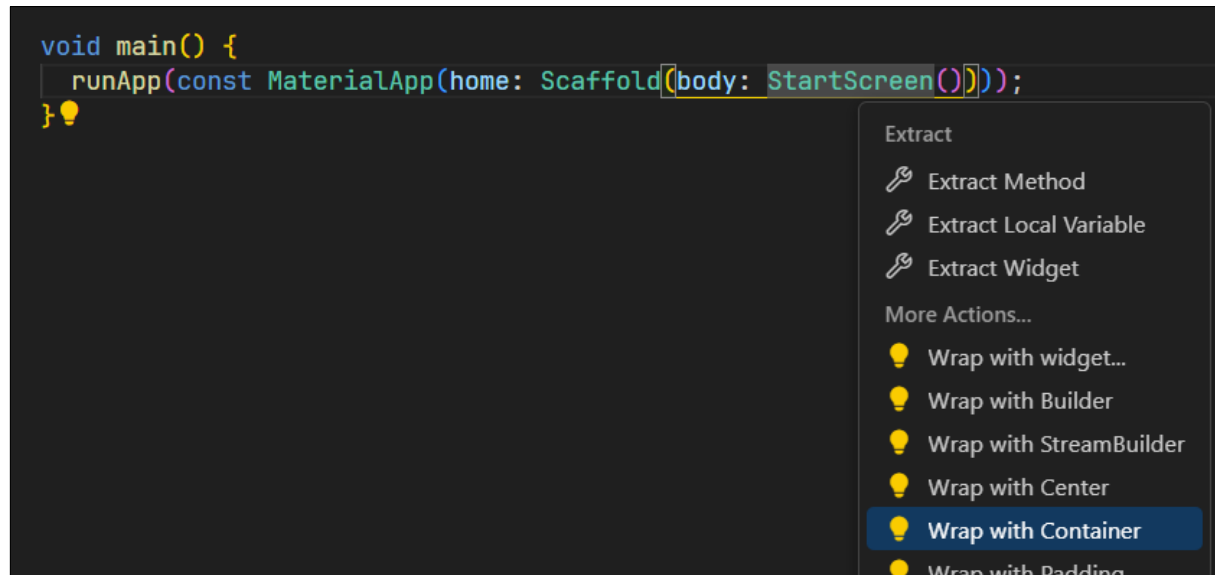
Fügt das StartScreen-Widget in die `main.dart`-Datei ein:

```
import 'package:flutter/material.dart';
import 'package:quiz_app/start_screen.dart';

void main() {
  runApp(const MaterialApp(home: Scaffold(body: StartScreen())));
}
```

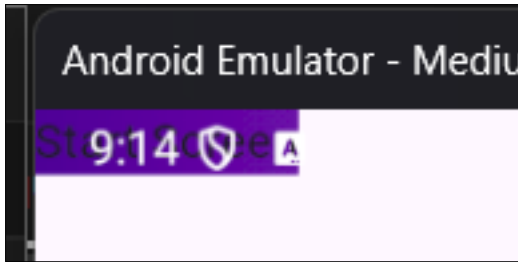
Ändern wir nun zuerst die Hintergrundfarbe.

Um eine Hintergrundfarbe zu setzen, betten wir das StartScreen-Widget in ein Container-Widget ein und setzen die Eigenschaften für die Hintergrundfarbe:



```
void main() {  
  runApp(MaterialApp(  
    home: Scaffold(  
      body: Container(  
        decoration: const BoxDecoration(  
          gradient: LinearGradient(colors: [  
            Color.fromARGB(255, 78, 13, 151),  
            Color.fromARGB(255, 107, 15, 168)  
          ])),  
        child: const StartScreen()))));  
}
```

Wenn wir nun die App starten, sieht sie in etwa so aus:



Der Inhalt unseres StartScreen-Widgets füllt bisher nur einen kleinen Teil des Bildschirms aus. Das wollen wir nun ändern:

```
class StartScreen extends StatelessWidget {  
  const StartScreen({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return const Center(child: Text('Start Screen'));  
  }  
}
```

Das Center-Widget zentriert nun den Text auf dem Bildschirm, da es standardmäßig versucht, den gesamten verfügbaren Platz zu füllen.

Ändern wir den Farbverlauf noch ein wenig ab:

```
gradient: LinearGradient(  
  colors: [  
    Color.fromARGB(255, 78, 13, 151),  
    Color.fromARGB(255, 107, 15, 168)  
  ],  
  begin: Alignment.topLeft,  
  end: Alignment.bottomRight),
```

Nun ersetzen wir den Dummy-Text durch den konkreten Inhalt unserer Startseite.

Zuerst wollen wir ein Bild anzeigen. Dazu müssen wir einen Eintrag in der Datei `pubspec.yaml` hinzufügen:

assets:

- assets/images/quiz-logo.png

Nun können wir das Bild in unser StartScreen-Widget einbauen:

```
class StartScreen extends StatelessWidget {  
  const StartScreen({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Center(  
      child: Column(  
        children: [Image.asset('assets/images/quiz-logo.png')],  
      )  
    );  
  }  
}
```

Das Bild wird nun horizontal zentriert angezeigt, ist aber noch zu groß:



Verkleinern wir es etwas und zentrieren außerdem auch alles vertikal:

@override

```
Widget build(BuildContext context) {  
  return Center(  
    child: Column(  
      mainAxisAlignment: MainAxisAlignment.min,  
      children: [  
        Image.asset(  
          'assets/images/quiz-logo.png',  
          width: 300,  
        )  
      ],  
    ),  
  );  
}
```

Für das Column-Widget setzen wir die Eigenschaft `mainAxisAlignment: MainAxisAlignment.center`, so dass alles vertikal zentriert wird.

Fügen wir nun noch einen Text und einen Button hinzu. So sieht die (vorerst) finale Version unseres StartScreen-Widgets aus:

```
class StartScreen extends StatelessWidget {  
  const StartScreen({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Center(  
      child: Column(  
        mainAxisAlignment: MainAxisAlignment.center,  
        children: [  
          Image.asset(  
            'assets/images/quiz-logo.png',  

```



```
        width: 300,
      ),
      SizedBox(height: 80),
      Text('Lern Flutter the fun way!',
        style: TextStyle(color: Colors.white, fontSize: 24)),
      SizedBox(height: 30),
      OutlinedButton(
        onPressed: () {},
        style: OutlinedButton.styleFrom(foregroundColor:
→ Colors.white),
        child: Text('Start Quiz'),
      )
    ],
  ),
);
}
```



Icon zu Button hinzufügen

Fügen wir nun ein Icon zu unserem Button hinzu. Wir verwenden dazu einen alternativen Konstruktor für das `OutlinedButton`-Widget. Außerdem setzen wir die Randfarbe noch auf Weiß:

```
OutlinedButton.icon(  
  onPressed: () {},  
  style: OutlinedButton.styleFrom(  
    foregroundColor: Colors.white,  
    iconColor: Colors.white,  
    side: BorderSide(width: 2.0, color: Colors.white),  
  ),  
  icon: const Icon(Icons.arrow_right_alt),  
  label: Text('Start Quiz'),  
)
```

Transparenz für ein Widget setzen

Nun möchten wir noch die Transparenz des Bildes etwas reduzieren. Dafür gibt es zwei Möglichkeiten. Die erste Möglichkeit besteht darin, das `Opacity`-Widget zu verwenden:

```
Opacity(  
  opacity: 0.8,  
  child: Image.asset(  
    'assets/images/quiz-logo.png',  
    width: 300,  
  ),  
)
```

Diese Variante ist aber nicht empfohlen, da sie sehr rechenintensiv ist. Besser ist es, die Transparenz direkt im Bild zu setzen:

```
Image.asset(  
  'assets/images/quiz-logo.png',  
  width: 300,  
  color: Color.fromRGBO(255, 255, 255, 0.8),  
  colorBlendMode: BlendMode.modulate,  
)
```

Man kann aber auch eine Farbe mit einem Transparenzwert verwenden:

```
color: Color.fromARGB(50, 255, 255, 255),  
//colorBlendMode: BlendMode.modulate,
```

Quiz-Widget erstellen

Wenn wir nun auf den Button klicken, soll das Quiz starten. Wir wollen also das StartScreen-Widget beim Klick durch ein neues QuestionsScreen-Widget ersetzen. Welches Widget wir anzeigen, hängt also von einem Zustand ab. Wir erstellen dazu also zuerst ein neues StatefulWidget namens Quiz:

```
import 'package:flutter/material.dart';  
import 'package:quiz_app/start_screen.dart';  
  
class Quiz extends StatefulWidget {  
  const Quiz({super.key});  
  
  @override  
  State<Quiz> createState() => _QuizState();  
}  
  
class _QuizState extends State<Quiz> {  
  @override
```

```
Widget build(BuildContext context) {  
  return MaterialApp(  
    home: Scaffold(  
      body: Container(  
        decoration: const BoxDecoration(  
          gradient: LinearGradient(colors: [  
            Color.fromARGB(255, 78, 13, 151),  
            Color.fromARGB(255, 107, 15, 168),  
          ], begin: Alignment.topLeft, end: Alignment.bottomRight),  
        ),  
        child: const StartScreen(),  
      ),  
    ),  
  );  
}
```

Wir übernehmen also alles, was bisher in der `main.dart`-Datei war, in das Quiz-Widget.

Nun erstellen wir noch ein zweites Widget, das die Fragen anzeigt. Dazu erstellen wir eine neue Datei `lib/questions_screen.dart`:

```
import 'package:flutter/material.dart';  
  
class QuestionsScreen extends StatefulWidget {  
  const QuestionsScreen({Key? key}) : super(key: key);  
  
  @override  
  State<QuestionsScreen> createState() => _QuestionsScreenState();  
}  
  
class _QuestionsScreenState extends State<QuestionsScreen> {
```

```
@override
Widget build(context) {
  return const Text('Questions Screen');
}
```

Zum Schluss können wir noch die `main.dart`-Datei anpassen:

```
void main() {
  runApp(Quiz());
}
```

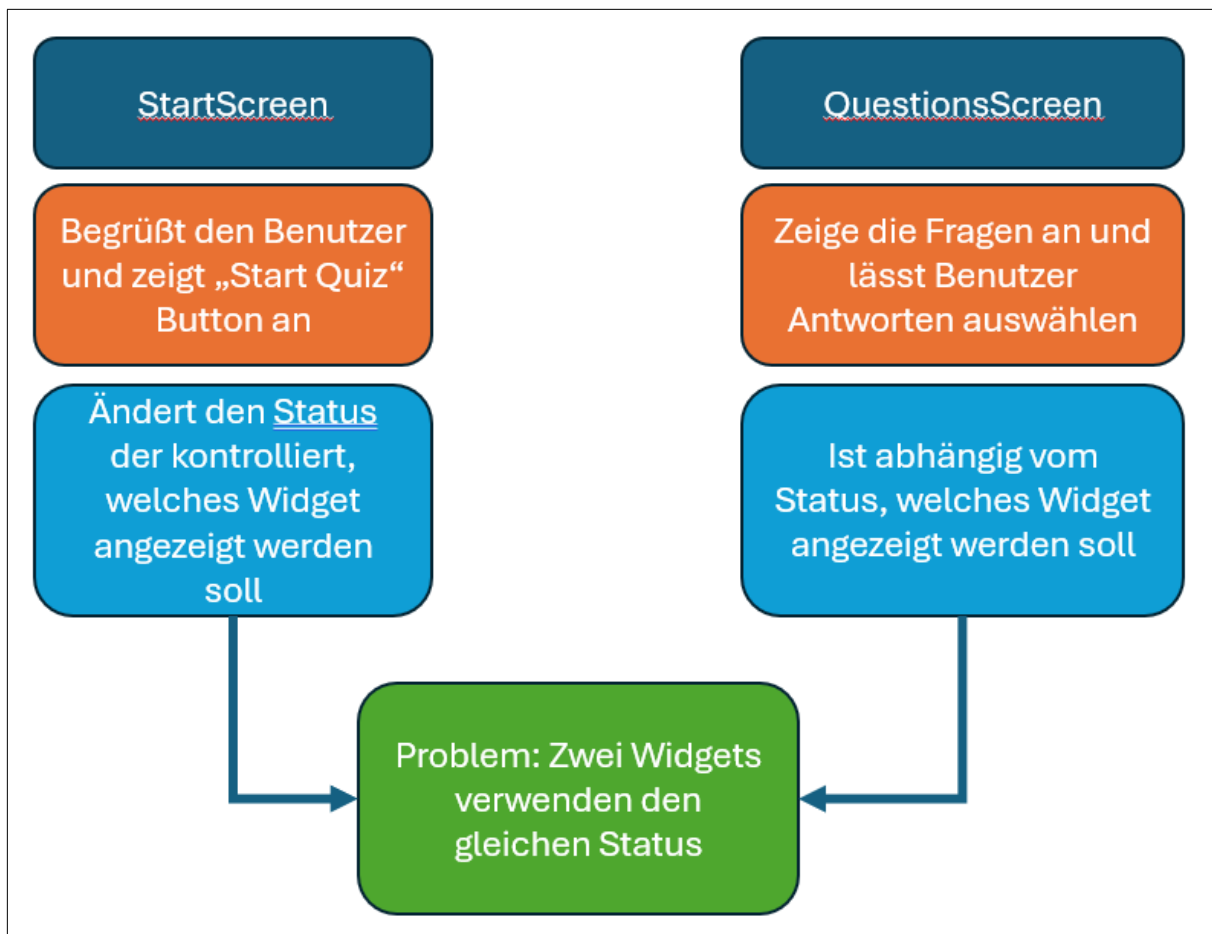
Wenn wir die App nun aktualisieren, sollten wir das gleiche Ergebnis haben wie zuvor. Nun können wir aber das `StartScreen`-Widget durch das `QuestionsScreen`-Widget ersetzen bei Bedarf, was wir im nächsten Schritt machen werden.

Zustandsverwaltung

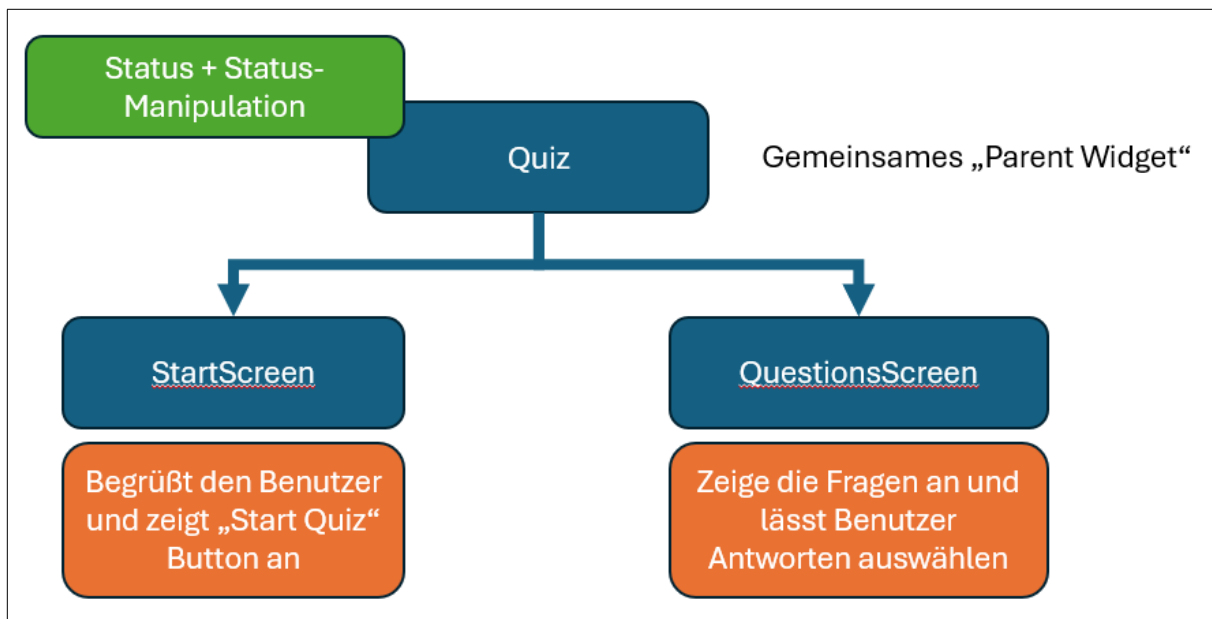
Nun sehen wir uns an, wie wir Inhalte abhängig von einem **Zustand** anzeigen können. Wir haben dieses Pattern schon bei der Würfel-App kennen gelernt: **Lifting State Up**.

Aktuell zeigen wir fix das `StartScreen`-Widget an. Wir wollen aber, dass beim Klick auf den Button das `QuestionsScreen`-Widget angezeigt wird. Dazu müssen wir den Zustand der App verwalten.

Wir haben also folgendes Problem:



Das lösen wir, indem wir den Zustand in ein übergeordnetes Widget verschieben - "**lifting state up**". Wir verschieben also den Zustand in das Quiz-Widget.



Erweitern wir das Quiz-Widget um einen Zustand:

```

...
class _QuizState extends State<Quiz> {
  Widget _screen = const StartScreen();

  void switchScreen() {
    setState(() {
      _screen = const QuestionsScreen();
    });
  }

  ...
  child: _screen,
  ...

```

Nun müssen wir noch sicherstellen, dass das StartScreen-Widget Zugriff auf die switchScreen()-Methode hat.

Wir machen wir das? Ganz einfach: Methoden sind **first-class objects** in Dart, sprich

sie können z.B. als **Argumente** übergeben werden - in unserem Fall im Konstruktor des StartScreen-Widgets:

```
class StartScreen extends StatelessWidget {  
  final void Function() startQuiz;  
  
  const StartScreen(this.startQuiz, {super.key});  
  
  ...  
  onPressed: startQuiz,  
  ...
```

Nun müssen wir nur noch die `switchScreen()`-Methode in der Konstruktor-Methode des StartScreen-Widgets übergeben:

```
...  
class _QuizState extends State<Quiz> {  
  Widget _screen = const StartScreen(switchScreen);  
  
  void switchScreen() {  
    setState(() {  
      _screen = const QuestionsScreen();  
    });  
  }  
  ...
```

Leider haben wir noch eine Fehlermeldung, da wir die Methode `switchScreen()` im Konstruktor des StartScreen-Widgets übergeben. Beide Bestandteile der `_QuizState`-Klasse sind aber noch nicht bekannt. Wie lösen wir dieses Problem?

Die `initState()`-Methode

Die `initState()`-Methode wird aufgerufen, wenn das State-Objekt zum ersten Mal erstellt wird. Das ist der richtige Zeitpunkt, um die Methode `switchScreen()` zu übergeben:

```
...  
class _QuizState extends State<Quiz> {  
  Widget? _screen;  
  
  @override  
  void initState() {  
    _screen = StartScreen(switchScreen);  
    super.initState();  
  }  
  
  void switchScreen() {  
    setState(() {  
      _screen = const QuestionsScreen();  
    });  
  }  
  ...  
}
```

Wichtige Lifecycle-Methoden von `StatefulWidget`

- `initState()`: Wird aufgerufen, wenn das State-Objekt zum ersten Mal erstellt wird.
- `build()`: Wird aufgerufen, wenn das Widget zum ersten Mal erzeugt wurde wenn sich der Zustand des Widgets ändert.
- `dispose()`: Wird aufgerufen, wenn das State-Objekt zerstört wird.

Denk-Aufgabe

Wie könnten wir das Umschalten zwischen den Screens noch anders lösen? Vielleicht sogar ohne die `initState()`-Methode?

Ein Datenmodell erstellen

Beginnen wir nun mit der Implementierung des Quiz-Widgets. Dazu erstellen wir zuerst ein Datenmodell für die Fragen. Erstelle dazu eine neue Datei `lib/models/quiz_question.dart`:

```
class QuizQuestion {  
  const QuizQuestion(this.text, this.answers);  
  
  final String text;  
  final List<String> answers;  
}
```

Die konkreten Frage und Antworten lagern wir in eine andere Datei `lib/data/questions.dart` aus:

```
import 'package:quiz_app/models/quiz_question.dart';  
  
const questions = [  
  QuizQuestion(  
    'What are the main building blocks of Flutter UIs?',  
    [  
      'Widgets',  
      'Components',  
      'Blocks',  
      'Functions',  
    ],  
  ),  
],
```

```
QuizQuestion('How are Flutter UIs built?', [
  'By combining widgets in code',
  'By combining widgets in a visual editor',
  'By defining widgets in config files',
  'By using XCode for iOS and Android Studio for Android',
]),
QuizQuestion(
  'What\'s the purpose of a StatefulWidget?',
  [
    'Update UI as data changes',
    'Update data as UI changes',
    'Ignore data changes',
    'Render UI that does not depend on data',
  ],
),
QuizQuestion(
  'Which widget should you try to use more often: StatelessWidget  
↔ or StatefulWidget?',
  [
    'StatelessWidget',
    'StatefulWidget',
    'Both are equally good',
    'None of the above',
  ],
),
QuizQuestion(
  'What happens if you change data in a StatelessWidget?',
  [
    'The UI is not updated',
    'The UI is updated',
```

```
        'The closest StatefulWidget is updated',
        'Any nested StatefulWidgets are updated',
    ],
),
QuizQuestion(
    'How should you update data inside of StatefulWidgets?',
    [
        'By calling setState()',
        'By calling updateData()',
        'By calling updateUI()',
        'By calling updateState()',
    ],
),
];
```

Nun bauen wir das QuestionsScreen-Widget um. Beginnen wir mit dem grundlegenden Aufbau:

```
import 'package:flutter/material.dart';

class QuestionsScreen extends StatefulWidget {
    const QuestionsScreen({Key? key}) : super(key: key);

    @override
    State<QuestionsScreen> createState() => _QuestionsScreenState();
}

class _QuestionsScreenState extends State<QuestionsScreen> {
    @override
    Widget build(context) {
        return SizedBox(
```

```
width: double.infinity,
child: Column(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    const Text(
      'The question...',
      style: TextStyle(color: Colors.white),
    ),
    SizedBox(height: 20),
    ElevatedButton(
      onPressed: () {},
      child: const Text('Answer 1'),
    ),
    ElevatedButton(
      onPressed: () {},
      child: const Text('Answer 2'),
    ),
    ElevatedButton(
      onPressed: () {},
      child: const Text('Answer 3'),
    ),
  ],
),
);
}
```

Wiederverwendbare Buttons erstellen

Für die Antworten möchten wir eine einheitliche Optik verwenden. Dazu erstellen wir ein neues StatelessWidget namens AnswerButton:

```
import 'package:flutter/material.dart';

class AnswerButton extends StatelessWidget {
  final String text;
  final void Function() onPressed;

  const AnswerButton(this.text, this.onPressed, {super.key});

  @override
  Widget build(BuildContext context) {
    return ElevatedButton(
      onPressed: onPressed,
      style: ElevatedButton.styleFrom(
        backgroundColor: Color.fromARGB(255,33,1,95),
        foregroundColor: Colors.white,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(40),
        ),
      ),
      child: Text(
        text,
        textAlign: TextAlign.center,
      ),
    );
  }
}
```

Nun können wir die `AnswerButton`-Widgets in unserem `QuestionsScreen`-Widget verwenden:

```
children: [
  ...
  const Text(
    'The question...',
    style: TextStyle(color: Colors.white),
  ),
  SizedBox(height: 20),
  AnswerButton(
    onPressed: () {},
    text: 'Answer 1',
  ),
  AnswerButton(
    onPressed: () {},
    text: 'Answer 2',
  ),
  AnswerButton(
    onPressed: () {},
    text: 'Answer 3',
  ),
],
...
```

Nun ersetzen wir noch die Dummy-Texte durch die konkreten Fragen und Antworten und passen das Styling noch etwas an:

```
...
import 'package:quiz_app/data/questions.dart';
...
class _QuestionsScreenState extends State<QuestionsScreen> {
```


@override

```
Widget build(context) {  
  final currentQuestion = questions[0]; // <--  
  return SizedBox(  
    width: double.infinity,  
    child: Container( // <--  
      margin: const EdgeInsets.all(40), // <--  
      child: Column(  
        mainAxisAlignment: MainAxisAlignment.center,  
        crossAxisAlignment: CrossAxisAlignment.stretch, // <--  
        children: [  
          Text(  
            currentQuestion.text, // <--  
            style: TextStyle(color: Colors.white),  
            textAlign: TextAlign.center, // <--  
          ),  
          SizedBox(height: 20),  
          ...currentQuestion.answers.map((answer) { // <--  
            return AnswerButton(  
              onPressed: () {},  
              text: answer,  
            );  
          }  
        ),  
        ],  
      ),  
    );  
  }  
}
```

...

Antworten durchmischen

Nun mischen wir noch die Antworten durch. Erweitern wir dazu unsere Model-Klasse um einen Getter:

```
class QuizQuestion {  
  const QuizQuestion(this.text, this.answers);  
  
  final String text;  
  final List<String> answers;  
  
  // List<String> getShuffledAnswers() {  
  //   final shuffledAnswers = List<String>.from(answers)..shuffle();  
  //   return shuffledAnswers;  
  // }  
  
  get shuffledAnswers => [...answers]..shuffle();  
}
```

HINWEIS: Die `shuffle()`-Methode verändert das ursprüngliche Array. Um das zu verhindern, erstellen wir eine Kopie des Arrays mit `List<String>.from(answers)`. Das `..`-Operator wird verwendet, um eine Methode auf das Ergebnis einer anderen Methode anzuwenden.

Das können wir nun in das `QuestionsScreen`-Widget einbauen:

...

```
...currentQuestion.shuffledAnswers.map((answer) {
```

...

Zur nächsten Frage springen

Nun wollen wir noch die nächste Frage anzeigen, wenn auf eine Antwort geklickt wird. Dazu müssen wir den Index der aktuellen Frage speichern und bei jedem Klick erhöhen:

```
...  
class _QuestionsScreenState extends State<QuestionsScreen> {  
  int _questionIndex = 0;  
  
  void _answerQuestion() {  
    setState() {  
      _questionIndex++;  
      _questionIndex = _questionIndex % questions.length;  
    };  
  }  
  
  @override  
  Widget build(context) {  
    final currentQuestion = questions[_questionIndex];  
    ...  
    AnswerButton(  
      onPressed: _nextQuestion,  
      text: answer,  
    );  
    ...  
  }  
}
```

Third-Party-Packages und Google-Fonts verwenden

Im folgenden Link wird ausführlich erklärt, wie man eigene Fonts in Flutter einbindet:

<https://docs.flutter.dev/cookbook/design/fonts>

Wie verwenden aber einen anderen Ansatz: Wir verwenden das Package `google_fonts`. Das Package `google_fonts` ermöglicht es, Google Fonts in Flutter zu verwenden, ohne die Schriftarten herunterladen und in das Projekt einbinden zu müssen. Rufen wir also folgenden Befehl auf, um das Package zu installieren:

```
flutter pub add google_fonts
```

HINWEIS: Weitere nützliche Packages für Flutter findest du unter <https://pub.dev/>

Nun können wir es in das `Text`-Widget einbauen wo wir es verwenden möchten. Du findest eine Übersicht über alle verfügbaren Google Fonts unter <https://fonts.google.com/>.

```
...
import 'package:google_fonts/google_fonts.dart'; // <--
...
child: Column(
  mainAxisAlignment: MainAxisAlignment.center,
  crossAxisAlignment: CrossAxisAlignment.stretch,
  children: [
    Text(
      currentQuestion.text,
      style: GoogleFonts.lato( // <--
        color: Colors.white,
        fontSize: 24,
      ),
      textAlign: TextAlign.center,
    ),
    ...
```

Nun sieht unsere App schon ziemlich gut aus:



Antworten speichern

Erweitern wir nun das Quiz-Widget so, dass wir dort die Antworten speichern können. Dies ist die korrekte Stelle, da dieses Widget den Zustand der App verwaltet, z.B. welcher Screen gerade angezeigt wird.

...

```
class _QuizState extends State<Quiz> {  
  final List<String> selectedAnswers = []; // <--
```

```
var activeScreen = 'start-screen';

void switchScreen() {
  setState(() {
    activeScreen = 'questions-screen';
  });
}

void chooseAnswer(String answer) {
  selectedAnswers.add(answer);
}

@override
Widget build(BuildContext context) {
  Widget screenWidget = StartScreen(switchScreen);

  if (activeScreen == 'questions-screen') {
    screenWidget = QuestionsScreen(
      onSelectAnswer: chooseAnswer, // <--
    );
  }
}
```

Wir übergeben der Konstruktor-Methode des QuestionsScreen-Widgets die Methode chooseAnswer(). Dazu müssen wir den Konstruktor entsprechend anpassen:

```
...
class QuestionsScreen extends StatefulWidget {
  final void Function(String) onSelectAnswer; // <--

  const QuestionsScreen({
    required this.onSelectAnswer, // <--
```

```
    super.key,  
  });  
  ...
```

Nun haben wir aber das Problem, dass die Methode `onSelectAnswer` in der `_QuestionsScreenState`-Klasse nicht bekannt ist. Wie lösen wir das Problem?

Antwort: Flutter stellt ein eigenes Property in der State-Klasse zur Verfügung, um auf die Eigenschaften des übergeordneten Widgets zuzugreifen: `widget`. Wir können also auf die Methode `onSelectAnswer` wie folgt zugreifen:

```
...  
class _QuestionsScreenState extends State<QuestionsScreen> {  
  int _questionIndex = 0;  
  
  void _answerQuestion(String answer) { // <--  
    widget.onSelectAnswer(answer); // <--  
    setState() {  
      _questionIndex++;  
      _questionIndex = _questionIndex % questions.length;  
    };  
  }  
  
  @override  
  Widget build(context) {  
    final currentQuestion = questions[_questionIndex];  
  
    return SizedBox(  
      width: double.infinity,  
      child: Container(  
        margin: const EdgeInsets.all(40),  
        child: Column(
```

```
mainAxisAlignment: MainAxisAlignment.center,
crossAxisAlignment: CrossAxisAlignment.stretch,
children: [
  Text(
    currentQuestion.text,
    style: GoogleFonts.lato(
      color: Colors.white,
      fontSize: 24,
    ),
    textAlign: TextAlign.center,
  ),
  SizedBox(height: 20),
  ...currentQuestion.shuffledAnswers.map((answer) {
    return AnswerButton(
      onPressed: () => _answerQuestion(answer), // <--
      text: answer,
    );
  }),
],
),
),
);
}
```

Wir erweitern also die Methode `_answerQuestion()` um den Parameter `answer` und rufen die Methode `onSelectAnswer(answer)` auf, um die Antwort zu speichern. Dazu ersetzen wir den direkten Aufruf bei `onPressed` durch eine anonyme Funktion, die die Methode `_answerQuestion(answer)` aufruft.

Übung

Erweitere die App so, dass wenn alle Fragen gestellt wurden, wieder die Startseite angezeigt wird.

Startseite anzeigen, wenn alle Fragen gestellt wurden

Dazu prüfen wir in der Methode `chooseAnswer()` im `Quiz`-Widget, ob alle Fragen gestellt wurden:

```
...  
class _QuizState extends State<Quiz> {  
  List<String> selectedAnswers = []; // <--  
  var activeScreen = 'start-screen';  
  
  void switchScreen() {  
    setState(() {  
      activeScreen = 'questions-screen';  
    });  
  }  
  
  void chooseAnswer(String answer) {  
    selectedAnswers.add(answer);  
  
    if (selectedAnswers.length == questions.length) { // <--  
      setState(() {  
        activeScreen = 'start-screen';  
        selectedAnswers = [];  
      });  
    }  
  }  
}
```

...

Seite mit Ergebnissen anzeigen

Beginnen wir nun mit der Ergebnis-Seite. Dazu erstellen wir wieder ein eigenes stateless Widget mit dem Namen ResultsScreen:

```
class ResultsScreen extends StatelessWidget {  
  const ResultsScreen({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return SizedBox(  
      width: double.infinity,  
      child: Container(  
        margin: const EdgeInsets.all(40),  
        child: Column(  
          mainAxisAlignment: MainAxisAlignment.center,  
          children: [  
            Text('You answered X out of Y questions correctly!'),  
            SizedBox(  
              height: 30,  
            ),  
            Text('List of answers and questions...'),  
            SizedBox(height: 30),  
            TextButton(  
              onPressed: () {},  
              child: Text('Restart Quiz!'),  
            )  
          ],  
        ),  
      ),  
    );  
  }  
}
```

```
        ),  
      ),  
    );  
  }  
}
```

Nun müssen wir noch anpassen, dass, wenn alle Fragen beantwortet wurden, die Ergebnis-Seite angezeigt wird. Dazu müssen wir den Zustand des Quiz-Widgets erweitern:

```
...  
void chooseAnswer(String answer) {  
  selectedAnswers.add(answer);  
  
  if (selectedAnswers.length == questions.length) {  
    setState(() {  
      activeScreen = 'results-screen'; // <--  
      selectedAnswers = [];  
    });  
  }  
}
```

@override

```
Widget build(BuildContext context) {  
  Widget screenWidget = StartScreen(switchScreen);  
  
  if (activeScreen == 'questions-screen') {  
    screenWidget = QuestionsScreen(  
      onSelectAnswer: chooseAnswer,  
    );  
  } else if (activeScreen == 'results-screen') { // <--  
    screenWidget = ResultsScreen();  
  }  
}
```

```
}  
...
```

Antworten an Ergebnis-Seite übergeben

Um die vom Benutzer gewählten Antworten an die Ergebnis-Seite zu übergeben, erweitern wir wieder den Konstruktor und somit auch den Aufruf im Quiz-Widget:

```
class ResultsScreen extends StatelessWidget {  
  final List<String> chosenAnswers;  
  
  const ResultsScreen({required this.chosenAnswers, super.key});  
  ...  
}
```

Quiz-Widget:

```
...  
} else if (activeScreen == 'results-screen') {  
  screenWidget = ResultsScreen(chosenAnswers: selectedAnswers);  
}  
...
```

Korrekte Antworten ermitteln und anzeigen

Nun müssen wir noch die korrekten Antworten ermitteln. Dazu erstellen wir eine Methode in der ResultsScreen-Klasse:

```
List<Map<String, Object>> getSummaryData() {  
  List<Map<String, Object>> summary = [];  
  
  for (var i = 0; i < chosenAnswers.length; i++) {  
    summary.add({
```

```
        'question_index': i,  
        'question': questions[i].text,  
        'correct_answer': questions[i].answers[0],  
        'user_answer': chosenAnswers[i],  
    });  
}  
  
    return summary;  
}
```

Um die Zusammenfassung im ResultsScreen-Widget anzuzeigen und das Widget nicht zu überladen, erstellen wir ein neues StatelessWidget namens QuestionsSummary:

```
class QuestionsSummary extends StatelessWidget {  
    final List<Map<String, Object>> summaryData;  
  
    const QuestionsSummary({required this.summaryData, super.key});  
  
    @override  
    Widget build(BuildContext context) {  
        return Column(  
            children: summaryData.map(  
                (data) {  
                    return Row(  
                        children: [  
                            Text(((data['question_index'] as int) + 1).toString()),  
                            Column(  
                                children: [  
                                    Text(data['question'] as String),  
                                    SizedBox(height: 5),  
                                    Text(data['user_answer'] as String),  
                                ],  
                            ),  
                        ],  
                    );  
                },  
            ),  
        );  
    }  
}
```

```

        Text(data['correct_answer'] as String),
      ],
    ),
  ],
);
},
).toList(),
);
}
}

```

Dieses neue Widget können wir nun in das ResultsScreen-Widget einbauen:

```

...
children: [
  Text('You answered X out of Y questions correctly!'),
  SizedBox(
    height: 30,
  ),
  QuestionsSummary(summaryData: getSummaryData()), // <--
  SizedBox(height: 30),
  ...

```

Wenn wir nun die App starten und alle Fragen beantwortet haben ... wird nichts angezeigt.
Warum?

Der Grund ist folgende Zeile in der Quiz-Klasse:

```

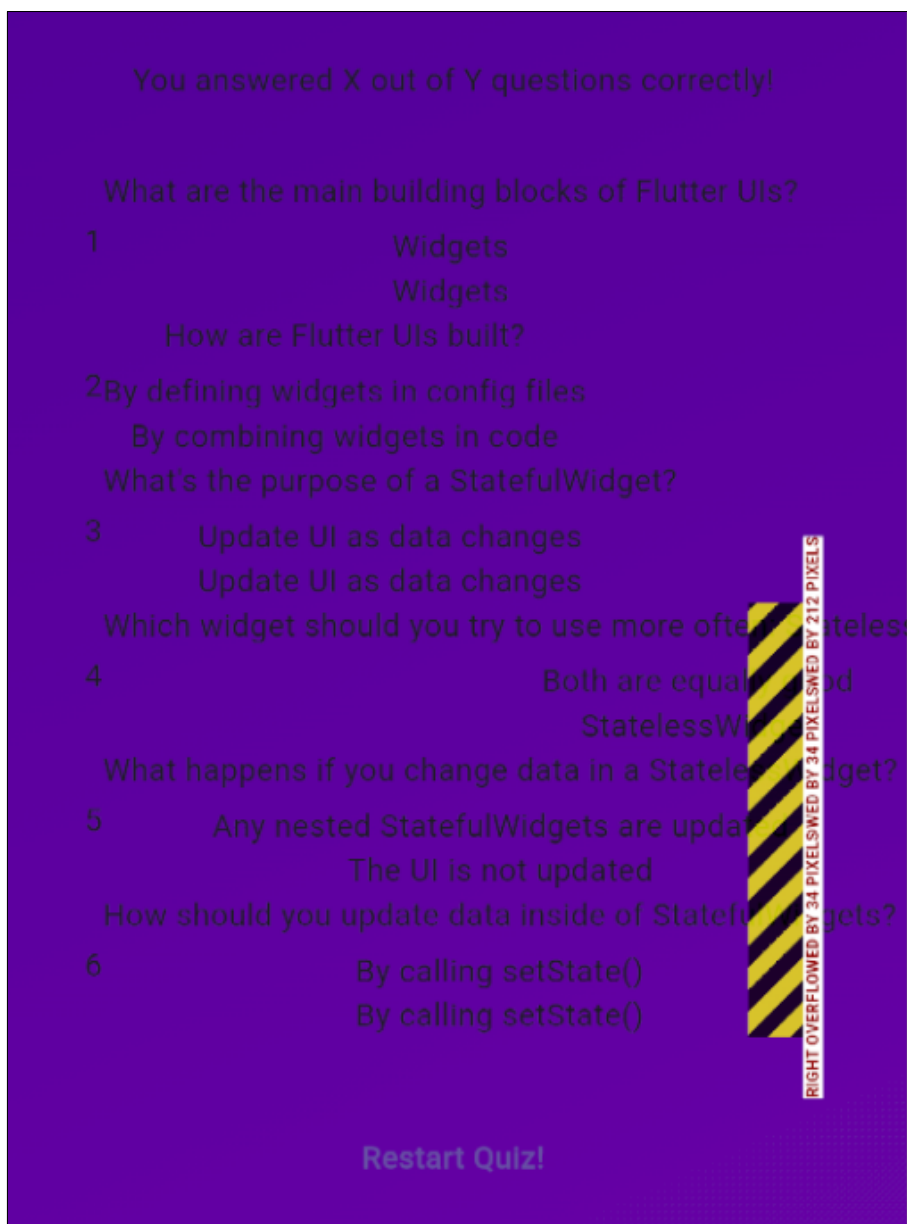
...
if (selectedAnswers.length == questions.length) {
  setState(() {
    activeScreen = 'results-screen';
    selectedAnswers = []; // <--

```

```
}));  
}  
...
```

Diese müssen wir vorerst entfernen, da wir an dieser Stelle ja nicht mehr das Quiz neu starten, sondern vorerst nur die Ergebnis-Seite anzeigen wollen.

Die Ausgabe, nachdem alle Antworten gegeben wurden, sollte nun in etwa so aussehen:



Es sieht also noch alles ziemlich "grauslich" aus, doch darum werden wir uns gleich kümmern.

Styling - Teil 1

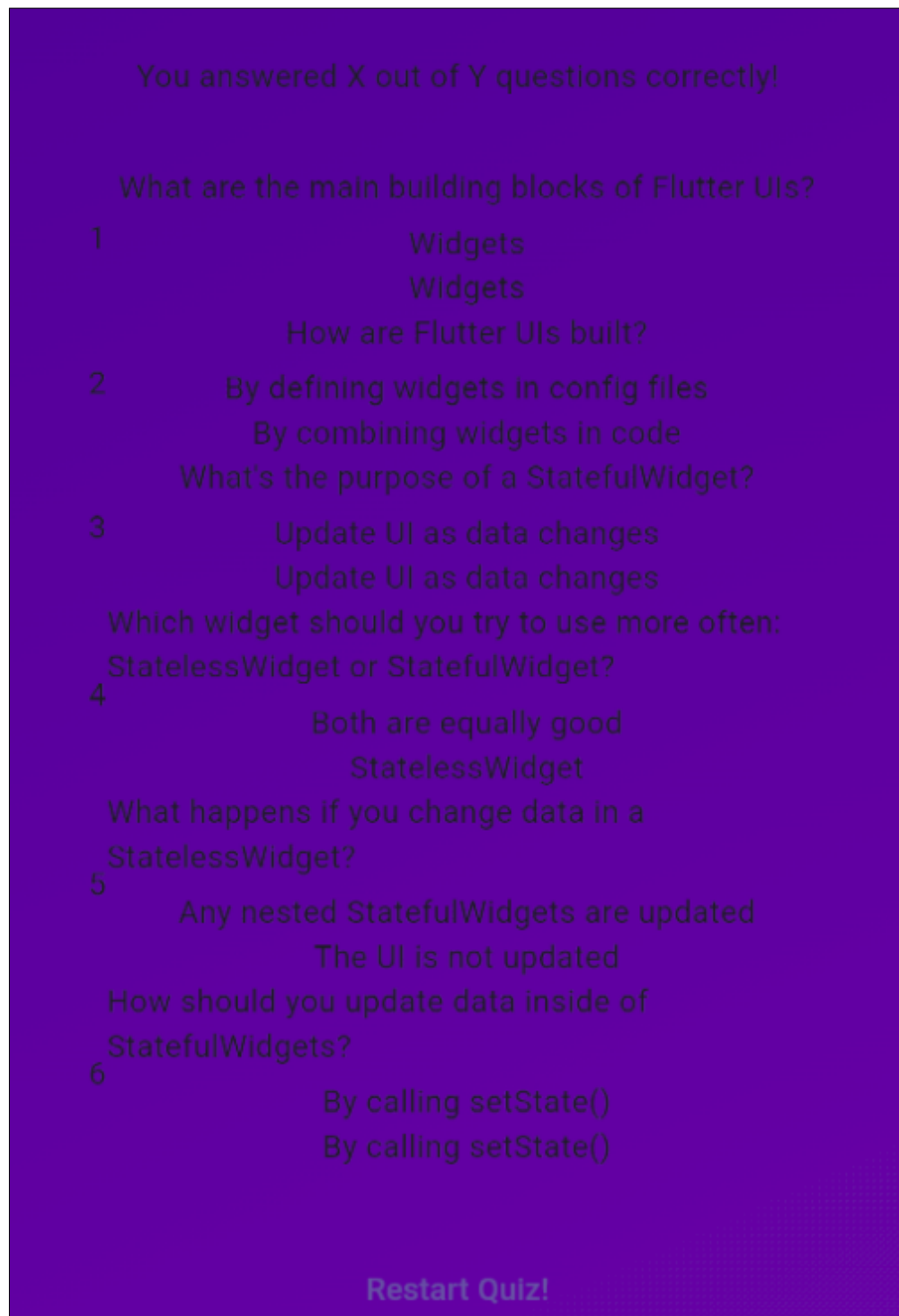
Kümmern wir uns zuerst um den Text, der abgeschnitten wird. Dazu betten wir den Text im QuestionsSummary-Widget in ein Expanded-Widget ein:

@override

```
Widget build(BuildContext context) {  
  return Column(  
    children: summaryData.map(  
      (data) {  
        return Row(  
          children: [  
            Text(((data['question_index'] as int) + 1).toString()),  
            Expanded( // <--  
              child: Column(  
                children: [  
                  Text(data['question'] as String),  
                  SizedBox(height: 5),  
                  Text(data['user_answer'] as String),  
                  Text(data['correct_answer'] as String),  
                ],  
              ),  
            ),  
          ],  
        ),  
      ),  
    ),  
  );  
}
```

Das Expanded-Widget sorgt dafür, dass der Text so viel Platz einnimmt, wie das Row-Widget zur Verfügung stellt.

Nun sieht das Ergebnis schon etwas besser aus:



Anzahl richtige Antworten ermitteln

Jetzt ist es an der Zeit, die Anzahl an richtigen Antworten herauszufinden. Dazu erweitern wir die ResultsScreen-Klasse wie folgt:

```
...  
@override  
Widget build(BuildContext context) {  
  final summaryData = getSummaryData(); // <--  
  final numTotalQuestions = questions.length; // <--  
  final numCorrectAnswers = summaryData.where((data) { // <--  
    return data['correct_answer'] == data['user_answer'];  
  }).length;  
  
  return SizedBox(  
    width: double.infinity,  
    child: Container(  
      margin: const EdgeInsets.all(40),  
      child: Column(  
        mainAxisAlignment: MainAxisAlignment.center,  
        children: [  
          Text(  
            'You answered $numCorrectAnswers out of  
            ↩ $numTotalQuestions questions correctly!'), // <--  
          ),  
          SizedBox(  
            height: 30,  
          ),  
          QuestionsSummary(summaryData: summaryData), // <--  
          SizedBox(height: 30),  
          ...  
        ],  
      ),  
    ),  
  );  
}
```

Zusammenfassung scrollbar machen

Um die Zusammenfassung scrollbar zu machen, betten wir das `QuestionsSummary-Widget` in ein `SingleChildScrollView-Widget` ein. Zuvor stellen wir aber sicher, dass das `Column-Widget` im `ResultsScreen-Widget` nicht mehr den gesamten verfügbaren Platz einnimmt:

...

@override

```
Widget build(BuildContext context) {  
  return SizedBox( // <--  
    height: 300,  
    child: SingleChildScrollView( // <--  
      child: Column(  
        children: summaryData.map(  
          (data) {  
            ...  
          }  
        )  
      )  
    )  
  }  
}
```

Nun können wir den Inhalt der Zusammenfassung scrollen:

You answered 0 out of 6 questions correctly!

How are Flutter UIs built?

2 By combining widgets in a visual editor

By combining widgets in code

What's the purpose of a StatefulWidget?

3 Render UI that does not depend on data

Update UI as data changes

Which widget should you try to use more often:

StatelessWidget or StatefulWidget?

4

None of the above

StatelessWidget

What happens if you change data in a

StatelessWidget?

5

The closest StatefulWidget is updated

The UI is not updated

How should you update data inside of

Restart Quiz!

Übung: Finalisierung der App

Finalisiere die App, damit sie wie folgt aussieht und auch von hier aus das Quiz neu gestartet werden kann:



Erstelle für die Nummer der Frage und die Darstellung, ob diese korrekt beantwortet wurde oder nicht (im Screenshot blau für korrekt und rosa für inkorrekt) ein eigene Widget - ebenso für die Frage und die Antworten.